

ESTRUCTURA DE COMPUTADORE

Tema 4: Problemas

Problema 1

- 1) Consider the following instruction: `and x7, x8, x9` de RISC-V.
 - a. What are the values of control signals generated by the control for this instruction?
 - b. Which resources (blocks) perform a useful function for this instruction?
 - c. Which resources (blocks) produce no output for this instruction? Which resources produce output that is not used?



Problema 1

- 1) Consider the following instruction: `and x7, x8, x9` de RISC-V.
 - a. What are the values of control signals generated by the control for this instruction?

The value of the signals is as follows:

RegWrite	ALUSrc	ALUoperation	MemWrite	MemRead	MemToReg
true	0	"and"	false	false	0

Mathematically, the `MemRead` control wire is a “don’t care”: the instruction will run correctly regardless of the chosen value. Practically, however, `MemRead` should be set to `false` to prevent causing a segment fault or cache miss.



Problema 1

- 1) Consider the following instruction: `and x7, x8, x9` de RISC-V.
 - b. Which resources (blocks) perform a useful function for this instruction?

Registers, ALUsrc mux, ALU, and the MemToReg mux.



Problema 1

- 1) Consider the following instruction: `and x7, x8, x9` de RISC-V.
 - c. Which resources (blocks) produce no output for this instruction? Which resources produce output that is not used?

All blocks produce some output. The outputs of DataMemory and ImmGen are not used.



Problema 2

2) Consider the following instruction mix:

R-type	I-type (non Id)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

- What fraction of all instructions use data memory?
- What fraction of all instructions use instruction memory?
- What fraction of all instructions use the sign extend?
- What is the sign extend doing during cycles in which its output is not needed?



Problema 2

2) Consider the following instruction mix:

R-type	I-type (non ld)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

a. What fraction of all instructions use data memory?

Memory access instructions

ld and sd

$$25 + 10 = 35\%$$



Problema 2

2) Consider the following instruction mix:

R-type	I-type (non Id)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

b. What fraction of all instructions use instruction memory?

All instruction must be fetched from instruction memory before it can be executed

100%



Problema 2

2) Consider the following instruction mix:

R-type	I-type (non Id)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

c. What fraction of all instructions use the sign extend?

Only R-type instructions do not use the Sign extender

$$28 + 25 + 10 + 11 + 2 = 76\%$$



Problema 2

2) Consider the following instruction mix:

R-type	I-type (non Id)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

d. What is the sign extend doing during cycles in which its output is not needed?

The sign extend produces an output during every cycle. If its output is not needed, it is simply ignored.



Problema 3

- 3) When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one signal wire to get “broken” and always register a logical 0. This is often called a “stuck-at-0” fault.
- Which instructions fail to operate correctly if the MemToReg wire is stuck at 0?
 - Which instructions fail to operate correctly if the ALUSrc wire is stuck at 0?



Problema 3

- 3) When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one signal wire to get “broken” and always register a logical 0. This is often called a “stuck-at-0” fault.
- a. Which instructions fail to operate correctly if the MemToReg wire is stuck at 0?

Only loads are broken. MemToReg is either 1 or “don’t care” for all other instructions.



Problema 3

- 3) When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one signal wire to get “broken” and always register a logical 0. This is often called a “stuck-at-0” fault.
- b. Which instructions fail to operate correctly if the ALUSrc wire is stuck at 0?

I-type, loads , stores are all broken.



Problema 4

- 4) In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: `0x00c6ba23`.
- What are the values of the ALU control unit's inputs for this instruction?
 - What is the new PC address after this instruction is executed? Highlight the path through which this value is determined.
 - For each mux, show the values of its inputs and outputs during the execution of this instruction. List values that are register outputs at Reg [xn].
 - What are the input values for the ALU and the two add units?
 - What are the values of all inputs for the registers unit?



Problema 4

- 4) In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: 0x00c6ba23.

0000 0000 1100 0110 1011 1010 0010 0011 two

opcode = 010 0011 = 0x23 = 32+3 = 35

S-Type

imm [11:5] = 0000 000

imm [4:0] = 1 0100 = 20

rs1 = 13 = 0 1101

rs2 = 12 = 0 1100

func3 = 011

Imm[11:5]	rs2	rs1	func3	Imm[4:0]	opcode
0000000	01100	01101	011	10100	0100011
0000 0000 1100 0110 1011 1010 0010 0011					
sd x12, 20(x13)					



Problema 4

- 4) In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: `0x00c6ba23`.
- What are the values of the ALU control unit's inputs for this instruction?

ALUop	ALU control lines
00	0010



Problema 4

- 4) In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: 0x00c6ba23.
- b. What is the new PC address after this instruction is executed? Highlight the path through which this value is determined.

The new PC is the old PC + 4. This signal goes from the PC, through the “PC + 4” adder, through the “branch” mux, and back to the PC.



Problema 4

- 4) In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: 0x00c6ba23.
- c. For each **mux**, show the values of its inputs and outputs during the execution of this instruction. List values that are register outputs at Reg [xn].

ALUsrc : Inputs: Reg[x12] and 0x0000000000000014 ;
Output: 0x0000000000000014

MemToReg : Inputs: Reg[x13] + 0x14 and <undefined> ; output: <undefined>

Branch: Inputs: PC+4 and PC+(IMMx2)



Problema 4

- 4) In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: 0x00c6ba23.
- d. What are the input values for the ALU and the two add units?

ALU inputs: Reg[x13] and 0x0000000000000014

PC + 4 adder inputs: PC and 4

Branch adder inputs: PC and 0x0000000000000028



Problema 4

- 4) In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: 0x00c6ba23.
- e. What are the values of all inputs for the registers unit?

rs1: Reg[x13]

rs2: Reg[x12]

rd: <undefined>



Problema 5

- 5) Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

I-Mem/ D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

“Register read” is the time needed after the rising clock edge for the new register value to appear on the output. This value applies to the PC only. “Register setup” is the amount of time a register's data input must be stable before the rising edge of the clock. This value applies to both the PC and Register File.

- What is the latency of an R-type instruction (i.e., how long must the clock period be to ensure that this instruction works correctly)?
- What is the latency of ld?
- What is the latency of sd?
- What is the latency of beq?
- What is the latency of an I-type instruction?
- What is the minimum clock period for this CPU?



Problema 5

- 5) Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

I-Mem/ D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

- a. What is the latency of an R-type instruction (i.e., how long must the clock period be to ensure that this instruction works correctly)?

$$\text{R-type : } 30 + 250 + 150 + 25 + 200 + 25 + 20 = 700\text{ps}$$



Problema 5

- 5) Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

I-Mem/ D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

- b. What is the latency of *ld*?

$$ld : 30 + 250 + 150 + 25 + 200 + 250 + 25 + 20 = 950 \text{ ps}$$



Problema 5

- 5) Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

I-Mem/ D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

- c. What is the latency of sd?

$$\text{sd} : 30 + 250 + 150 + 200 + 25 + 250 = 905$$



Problema 5

- 5) Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

I-Mem/ D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

d. What is the latency of beq?

$$\text{beq} : 30 + 250 + 150 + 25 + 200 + 5 + 25 + 20 = 705$$



Problema 5

- 5) Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

I-Mem/ D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

- e. What is the latency of an I-type instruction?

$$\text{I-type : } 30 + 250 + 150 + 25 + 200 + 25 + 20 = 700\text{ps}$$



Problema 5

- 5) Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

I-Mem/ D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

- f. What is the minimum clock period for this CPU?

$$950 \text{ ps} = 1,05 \text{ GHz}$$

Problema 6

- 6) In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF: 250 ps; ID: 350 ps; EX: 150 ps; MEM: 300 ps; WB: 200 ps

Also, assume that instructions executed by the processor are broken down as follows:

ALU/Logic: 45%; Jump/Branch: 20%; Load: 20%; Store: 15

- a. What is the clock cycle time in a pipelined and non-pipelined processor?
- b. What is the total latency of an `ld` instruction in a pipelined and non-pipelined processor?
- c. If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?
- d. Assuming there are no stalls or hazards, what is the utilization of the data memory?
- e. Assuming there are no stalls or hazards, what is the utilization of the write-register port of the “Registers” unit?



Problema 6

- 6) In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF: 250 ps; ID: 350 ps; EX: 150 ps; MEM: 300 ps; WB: 200 ps

Also, assume that instructions executed by the processor are broken down as follows:

ALU/Logic: 45%; Jump/Branch: 20%; Load: 20%; Store: 15

- a. What is the clock cycle time in a pipelined and non-pipelined processor?

Pipelined: 350 ps

Non-pipelined: 1250 ps



Problema 6

- 6) In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF: 250 ps; ID: 350 ps; EX: 150 ps; MEM: 300 ps; WB: 200 ps

Also, assume that instructions executed by the processor are broken down as follows:

ALU/Logic: 45%; Jump/Branch: 20%; Load: 20%; Store: 15

- b. What is the total latency of an `ld` instruction in a pipelined and non-pipelined processor?

Pipelined: 1250

Non-pipelined: 1250



Problema 6

- 6) In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF: 250 ps; ID: 350 ps; EX: 150 ps; MEM: 300 ps; WB: 200 ps

Also, assume that instructions executed by the processor are broken down as follows:

ALU/Logic: 45%; Jump/Branch: 20%; Load: 20%; Store: 15

- c. If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?

Split the ID stage. This reduces the clock-cycle time to 300ps.



Problema 6

- 6) In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF: 250 ps; ID: 350 ps; EX: 150 ps; MEM: 300 ps; WB: 200 ps

Also, assume that instructions executed by the processor are broken down as follows:

ALU/Logic: 45%; Jump/Branch: 20%; Load: 20%; Store: 15

- d. Assuming there are no stalls or hazards, what is the utilization of the data memory?

35%



Problema 6

- 6) In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF: 250 ps; ID: 350 ps; EX: 150 ps; MEM: 300 ps; WB: 200 ps

Also, assume that instructions executed by the processor are broken down as follows:

ALU/Logic: 45%; Jump/Branch: 20%; Load: 20%; Store: 15

- e. Assuming there are no stalls or hazards, what is the utilization of the write-register port of the “Registers” unit?

65%



Problema 7

7) Consider the fragment of RISC-V assembly below:

```
sd x29, 12(x16)
ld x29, 8(x16)
sub x17, x15, x14
beqz x17, label
add x15, x11, x14
sub x15, x30, x14
```

Suppose we modify the pipeline so that it has only one memory (that handles both instructions and data). In this case, there will be a structural hazard every time a program needs to fetch an instruction during the same cycle in which another instruction accesses data.

- Draw a pipeline diagram to show where the code above will stall.
- In general, is it possible to reduce the number of stalls/NOPs resulting from this structural hazard by reordering code?
- Must this structural hazard be handled in hardware? We have seen that data hazards can be eliminated by adding NOPs to the code. Can you do the same with this structural hazard? If so, explain how. If not, explain why not.



Problema 7

7) Consider the fragment of RISC-V assembly below:

```
sd x29, 12(x16)
ld x29, 8(x16)
sub x17, x15, x14
beqz x17, label
add x15, x11, x14
sub x15, x30, x14
```

a. Draw a pipeline diagram to show where the code above will stall.

Stalls are marked with **:

sd x29, 12(x16)	IF	ID	EX	ME	WB													
ld x29, 8(x16)		IF	ID	EX	ME	WB												
sub x17, x15, x14			IF	ID	EX	ME	WB											
beqz x17, label				**	**	IF	ID	EX	ME	WB								
add x15, x11, x14							IF	ID	EX	ME	WB							
sub x15, x30, x14								IF	ID	EX	ME	WB						

Problema 7

7) Consider the fragment of RISC-V assembly below:

```
sd x29, 12(x16)
ld x29, 8(x16)
sub x17, x15, x14
beqz x17, label
add x15, x11, x14
sub x15, x30, x14
```

- b. In general, is it possible to reduce the number of stalls/NOPs resulting from this structural hazard by reordering code?

Reordering code won't help. Every instruction must be fetched; thus, every data access causes a stall. Reordering code will just change the pair of instructions that are in conflict.



Problema 7

7) Consider the fragment of RISC-V assembly below:

```
sd x29, 12(x16)
ld x29, 8(x16)
sub x17, x15, x14
beqz x17, label
add x15, x11, x14
sub x15, x30, x14
```

- c. Must this structural hazard be handled in hardware? We have seen that data hazards can be eliminated by adding NOPs to the code. Can you do the same with this structural hazard? If so, explain how. If not, explain why not.

You can't solve this structural hazard with NOPs, because even the NOPs must be fetched from instruction memory.

